



Quiz — Programação Funcional

Paradigmas de Linguagens de Programação · material de apoio do grupo

18 QUESTÕES · 6 FÁCEIS · 6 MÉDIAS · 6 DIFÍCEIS

Todas as questões foram verificadas contra os 16 slides da apresentação: **nada é cobrado sem ter sido explicado**. O mapa de cobertura está na última página.

Como montar no Wayground

Opção A — quiz único (recomendado): Create → *Import worksheets/questions* → **Paste questions** → cole os três blocos na ordem (fácil → médio → difícil) → *Generate Questions*. A dificuldade crescente mantém a turma engajada: todos acertam o começo e ganham confiança.

Opção B — três quizzes separados: cole um bloco por vez. Útil para usar o bloco fácil como aquecimento antes da aula e os outros dois no fim.

Tempo sugerido: fácil 15s · média 25s · difícil 35s por questão → o quiz completo roda em **~8 minutos**. A resposta correta está marcada com ✓ — confira após gerar, pois a IA do Wayground às vezes precisa de ajuste.

BLOCO FÁCIL — recall direto do que está na tela

- 1) O paradigma funcional tem origem em qual formalismo matemático da década de 1930?

A) Máquina de Turing

B) Cálculo Lambda de Alonzo Church ✓

C) Álgebra booleana de George Boole

D) Teoria dos grafos de Euler
- 2) O que caracteriza uma FUNÇÃO PURA?

A) Ela sempre altera variáveis globais para funcionar

B) Para o mesmo input, produz sempre o mesmo output, sem efeitos colaterais ✓

C) Ela depende do horário do sistema para retornar valores

D) Ela precisa fazer uma requisição HTTP a cada chamada
- 3) O princípio da IMUTABILIDADE afirma que:

A) Um dado, uma vez criado, não é alterado; mudanças geram novos dados ✓

B) Toda variável deve ser alterada pelo menos uma vez

C) Objetos devem ser sempre modificados no lugar para poupar memória

D) Só é permitido usar variáveis do tipo constante inteira
- 4) Qual linguagem, criada por John McCarthy em 1958, foi a primeira a levar o Cálculo Lambda para a prática?

A) Haskell

B) Fortran

C) LISP ✓

D) JavaScript
- 5) O WhatsApp lida com bilhões de conexões simultâneas apoiando-se em qual linguagem funcional, conhecida por tolerância a falhas?

A) Python

B) Erlang ✓

C) C

D) PHP
- 6) Na programação funcional, o que substitui os laços tradicionais (for e while) como mecanismo de repetição?

A) A recursão ✓

B) O comando goto

C) Threads paralelas

D) Ponteiros de memória

BLOCO MÉDIO — conectar conceito e exemplo

- 7) O que a expressão `λx. x + 1` representa no Cálculo Lambda?

A) Uma variável global mutável

B) Um laço de repetição infinito

C) Uma função anônima que recebe x e retorna seu sucessor ✓

D) Uma estrutura condicional

8) Uma **FUNÇÃO DE ALTA ORDEM** (higher-order) é aquela que:

- A) Executa mais rápido que as demais
- B) Recebe outras funções como parâmetro e/ou retorna uma função ✓**
- C) Só pode ser escrita em Haskell
- D) Não possui parâmetros

9) Qual alternativa descreve corretamente o que cada operação do "trio de ouro" devolve?

- A) map devolve um valor único; filter transforma; reduce seleciona
- B) map devolve lista do mesmo tamanho; filter devolve lista de tamanho menor ou igual; reduce devolve um único valor ✓**
- C) Os três sempre devolvem listas do mesmo tamanho da original
- D) Os três sempre devolvem um único valor acumulado

10) A expressão `(f ◦ g)(x) = f(g(x))` representa qual conceito, base dos famosos "pipelines de dados"?

- A) Recursão de cauda
- B) Composição de funções ✓**
- C) Herança múltipla
- D) Sobrecarga de operadores

11) Toda função recursiva precisa obrigatoriamente de quais duas peças?

- A) Um laço de repetição e um contador
- B) Um caso base que encerra e um passo que reduz o problema ✓**
- C) Uma variável global e um controle de concorrência
- D) Duas funções puras distintas

12) Por que a programação funcional é considerada segura para concorrência?

- A) Porque cria automaticamente mais threads
- B) Porque sem estado mutável compartilhado não ocorrem race conditions ✓**
- C) Porque bloqueia o acesso dos outros núcleos ao processador
- D) Porque só permite execução em um único núcleo

BLOCO DIFÍCIL — raciocínio e análise

13) Em qual situação a TRANSPARÊNCIA REFERENCIAL NÃO pode ser garantida?

- A) Quando a função é pura e usa apenas seus parâmetros
- B) Quando a função lê um estado global ou consulta um banco que pode ter mudado ✓**
- C) Quando a função recebe outra função como argumento
- D) Quando a função é usada dentro de um map

14) Observe o código: `numeros.filter(x => x % 2 === 0).map(x => x * 2)`. Qual afirmação está correta?

- A) É imperativo, pois controla o laço explicitamente
- B) É declarativo: encadeia funções de alta ordem, sem mutação, e a lista original permanece intacta ✓**
- C) Modifica o array `numeros` no próprio lugar para economizar memória
- D) Só funciona se `numeros` for uma variável global

15) Qual é o papel da otimização de chamada de cauda (tail call optimization)?

- A) Aumentar a quantidade de chamadas recursivas por segundo
- B) Quando a chamada recursiva é a última operação, o compilador reaproveita o espaço da pilha, evitando o estouro ✓**
- C) Converter qualquer função em pura automaticamente
- D) Eliminar a necessidade do caso base

16) Sobre o custo de memória da imutabilidade, é correto afirmar que:

- A) Não existe custo algum, pois nada é copiado
- B) Gerar novas versões pode elevar o uso de memória; estruturas persistentes reduzem isso compartilhando as partes que não mudaram ✓**
- C) O custo é resolvido apagando automaticamente os dados originais
- D) A imutabilidade sempre torna o programa mais rápido que um laço

17) No pipeline `filter → map → reduce` do relatório de faturamento, o que acontece com a lista original de pedidos ao final da execução?

- A) É esvaziada para liberar memória
- B) Permanece intacta, pois cada etapa gera novos dados em vez de alterar os existentes ✓**
- C) É reordenada por valor decrescente
- D) Passa a conter os valores já com imposto aplicado

18) O que significa "isolar os efeitos colaterais na borda" do sistema?

- A) Eliminar completamente qualquer operação de entrada e saída
- B) Concentrar banco, rede e arquivo numa camada fina de entrada/saída, mantendo pura a regra de negócio ✓**
- C) Colocar todos os efeitos colaterais dentro de funções puras
- D) Usar variáveis globais para centralizar o estado do programa

Gabarito rápido

Bloco	Questões	Respostas
Fácil	1-6	1-B · 2-B · 3-A · 4-C · 5-B · 6-A
Médio	7-12	7-C · 8-B · 9-B · 10-B · 11-B · 12-B
Difícil	13-18	13-B · 14-B · 15-B · 16-B · 17-B · 18-B

O gabarito ficou com muitos "B" por coincidência da montagem. Ao revisar no Wayground, ative o **embaralhamento de alternativas** para que a turma não perceba o padrão.

Mapa de cobertura — onde cada questão é ensinada

Q	Nível	Slide que ensina	Apresentador
1	Fácil	3 · Cálculo Lambda	Pessoa 1
2	Fácil	5 · Funções Puras	Pessoa 2
3	Fácil	5 · Imutabilidade	Pessoa 2
4	Fácil	4 · Linha do tempo	Pessoa 1
5	Fácil	14 · Mercado e indústria	Pessoa 4
6	Fácil	9 · Recursão	Pessoa 3
7	Médio	3 · Cálculo Lambda	Pessoa 1
8	Médio	7 · Alta ordem	Pessoa 2
9	Médio	11 · Map / Filter / Reduce	Pessoa 3
10	Médio	8 · Composição	Pessoa 2
11	Médio	9 · Recursão	Pessoa 3
12	Médio	13 · Prós e contras	Pessoa 4
13	Difícil	6 · Transparência referencial	Pessoa 2
14	Difícil	10 · Imperativo vs. funcional	Pessoa 3
15	Difícil	9 · Recursão (tail call)	Pessoa 3
16	Difícil	13 · Prós e contras	Pessoa 4
17	Difícil	12 · Caso de uso real	Pessoa 3
18	Difícil	15 · Como aplicar já	Pessoa 4

Distribuição por apresentador: Pessoa 1 → 3 questões · Pessoa 2 → 5 · Pessoa 3 → 6 · Pessoa 4 → 4.

⚠ Dois pontos que precisam ser ditos em voz alta

Todo o resto está escrito na face dos slides. Estes dois estão **apenas nas notas do apresentador** — se não forem falados, as questões 14 e 16 ficam sem base:

- A palavra **"declarativo"** (slide 10, Pessoa 3). O slide mostra "o que fazer" e "declaramos a intenção", mas o termo técnico só aparece na fala. → questão 14
- "Estruturas persistentes"** (slide 13, Pessoa 4). O slide traz só "uso de memória: imutabilidade pode gerar cópias em excesso"; a explicação de que linguagens modernas compartilham as partes que não mudaram está na fala. → questão 16